

HTTP и его версии. Методы GET, PUT, POST, DELETE

HTTP (Hypertext Transfer Protocol) — это основной протокол прикладного уровня, который используется для передачи данных в сети, в частности в интернете. Он определяет правила взаимодействия между клиентами (обычно веб-браузерами) и серверами, которые обрабатывают запросы и предоставляют ресурсы, такие как HTML-страницы, изображения и файлы.

Версии HTTP

HTTP прошел через несколько версий, каждая из которых привнесла улучшения в производительность, безопасность и функциональность.

HTTP/1.0

- **Описание:** Первая версия HTTP, выпущенная в 1996 году.
- **Особенности:**
 - Каждый запрос требует отдельного соединения между клиентом и сервером.
 - Соединение закрывается после обработки одного запроса.
 - Поддерживает только метод GET (хотя в практике позже добавили POST и HEAD).
 - Ограниченная поддержка кэширования.

HTTP/1.1

- **Описание:** Вышла в 1997 году и стала основной версией протокола на многие годы.
- **Особенности:**
 - **Постоянные соединения:** Разрешает использовать одно соединение для нескольких запросов/ответов, что значительно снижает накладные расходы на установку соединений.
 - **Улучшенное кэширование:** Включает заголовки, такие как Cache-Control, для управления кэшированием.
 - **Поддержка дополнительных методов:** Включает методы PUT, DELETE, OPTIONS, TRACE и CONNECT.
 - **Поддержка чанков (Chunked Transfer Encoding):** Разрешает передачу данных по частям, что полезно для динамически генерируемого контента.
 - **Улучшенное управление ошибками:** Включает больше кодов состояния для различных ситуаций.

HTTP/2

- **Описание:** Стандартизирован в 2015 году как замена HTTP/1.1.
- **Особенности:**
 - **Мультиплексирование:** Позволяет отправлять несколько запросов одновременно по одному соединению без блокировки.
 - **Сжатие заголовков (HPACK):** Уменьшает размер HTTP-заголовков для повышения производительности.
 - **Server Push:** Сервер может отправить дополнительные ресурсы, которые могут понадобиться клиенту, до того, как клиент их запросит.
 - **Бинарный формат:** Использует бинарный формат вместо текстового, что делает передачу данных более быстрой и эффективной.

HTTP/3

- **Описание:** Текущая версия HTTP, стандартизированная в 2020 году. Основана на протоколе **QUIC** (Quick UDP Internet Connections).
- **Особенности:**
 - **Использует UDP вместо TCP:** Устраняет некоторые недостатки TCP, такие, как высокая задержка при установлении соединений.
 - **Быстрое установление соединения:** Меньше времени на установку и возобновление соединений.
 - **Повышенная безопасность:** Все соединения по умолчанию защищены с использованием TLS 1.3.

Методы HTTP

HTTP предоставляет несколько методов запроса, которые указывают, какое действие клиент хочет выполнить с ресурсом на сервере.

1. Метод GET

- **Описание:** Запрашивает представление ресурса, не изменяя его на сервере.
- **Использование:**
 - Получение данных с сервера (например, загрузка веб-страницы или изображения).
 - Запрос не должен иметь побочных эффектов и не изменяет состояние сервера.
- **Пример запроса:**

```
GET /index.html HTTP/1.1
Host: www.example.com
```

- **Пример в Go:**

```
resp, err := http.Get("https://api.example.com/data")
if err != nil {
    log.Fatalln(err)
}
defer resp.Body.Close()
body, err := ioutil.ReadAll(resp.Body)
fmt.Println(string(body))
```

2. Метод POST

- **Описание:** Отправляет данные на сервер для создания или обновления ресурса.
- **Использование:**
 - Отправка формы на веб-сайте.
 - Передача данных на сервер для обработки (например, добавление нового пользователя в базу данных).
 - Метод POST может изменять состояние сервера и не является идиempотентным (повторные вызовы могут иметь разные эффекты).
- **Пример запроса:**

```
POST /submit-form HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 27

name=John&age=30
```

- **Пример в Go:**

```
data := url.Values{}
data.Set("name", "John")
data.Set("age", "30")

resp, err := http.PostForm("https://api.example.com/form", data)
if err != nil {
    log.Fatalln(err)
}
defer resp.Body.Close()
body, err := ioutil.ReadAll(resp.Body)
fmt.Println(string(body))
```

3. Метод PUT

- **Описание:** Заменяет текущий ресурс данными, отправленными в запросе.
- **Использование:**
 - Обновление существующего ресурса или создание нового, если ресурс не существует.
 - Идиempотентный метод (повторные вызовы приводят к одному и тому же результату).
- **Пример запроса:**

```
PUT /user/123 HTTP/1.1
Host: www.example.com
Content-Type: application/json
Content-Length: 26

{"name": "John", "age": 30}
```

- **Пример в Go:**

```
client := &http.Client{}
req, err := http.NewRequest(http.MethodPut, "https://api.example.com/user/123", bytes.NewBuffer([]byte(`{"name": "John", "age": 30}`)))
req.Header.Set("Content-Type", "application/json")
resp, err := client.Do(req)
if err != nil {
    log.Fatalln(err)
}
defer resp.Body.Close()
body, err := ioutil.ReadAll(resp.Body)
fmt.Println(string(body))
```

4. Метод DELETE

- **Описание:** Удаляет указанный ресурс на сервере.
- **Использование:**
 - Удаление существующего ресурса (например, удаление пользователя из базы данных).
 - Идиempотентный метод (повторные вызовы приводят к одному и тому же результату).
- **Пример запроса:**

```
DELETE /user/123 HTTP/1.1
Host: www.example.com
```

- **Пример в Go:**

```
client := &http.Client{}
req, err := http.NewRequest(http.MethodDelete, "https://api.example.com/user/123", nil)
resp, err := client.Do(req)
if err != nil {
    log.Fatalf(err)
}
defer resp.Body.Close()
body, err := ioutil.ReadAll(resp.Body)
fmt.Println(string(body))
```

Заключение

HTTP является основой для передачи данных в интернете и широко используется для создания веб-приложений. Различные версии HTTP вносят свои улучшения в производительность и безопасность. Методы HTTP, такие как GET, POST, PUT и DELETE, позволяют управлять данными и взаимодействовать с ресурсами на серверах, что делает HTTP чрезвычайно гибким и мощным инструментом для веб-разработчиков.